# Why the tasks fail

OfficeBench · gemma4:31b (Ollama, Thought-then-Command prompt, run tag *think*) · snapshot 2026-10-02 · 73 of 300 subtasks scored, all single-app

## Summary

Of the 73 subtasks scored so far, **52 pass and 21 fail (71.2% pass)**. I read every failed run step by step (the model's thought, the action, the observation), compared the final files with the task text and the reference files, and re-ran each check. The main finding is that **most failures are not the model's fault**:

- **10 failures are on the benchmark side.** The agent did what the task says, but a tool bug, a check that asks for something the task never mentions, or a formatting rule marked it wrong.
- **4 failures have a broken check *and* an agent error.** The agent made a mistake, but the check is also broken (a misspelled reference file, a wrong reference, a wrong expected number), so a perfect agent would still fail.
- **Only 7 failures are clean agent errors**, where the benchmark is sound and the model got it wrong.
- In total, **at least 8 of the 73 scored subtasks cannot be passed by an agent that does exactly what the task says** (1-3/4, 1-5/1, 1-6/0, 1-6/3, 1-8/0, 1-9/0, 1-14/2, 1-14/3).
- The agent's own errors fall into a few patterns: it does a different operation from the one asked (merging into an existing row, swapping only the labels, averaging the wrong thing), it ignores a conflict it noticed, it re-does arithmetic at the next step and gets it wrong, and it runs out of the 20-step budget because the Excel app writes only one cell per action.

**Note on the dashboard count.** The dashboard shows 22 failures. The extra one, 1-16/0, is a false failure: the local evaluator reads the task JSON with the Windows default encoding (cp1252), so the expected keyword "café" becomes "cafÃ©" and never matches. With UTF-8 the check passes. This report uses the UTF-8 result (52 pass / 21 fail).

## Where the 21 failures come from

| Side | Cause | n | Subtasks |
|---|---|---|---|
| Benchmark | Excel tool silently erases any cell written as "1" | 2 | 1-5/1, 1-6/0 |
| Benchmark | Check requires something the task does not ask for, or reads a different file | 7 | 1-2/0, 1-2/2, 1-6/3, 1-8/0, 1-8/2, 1-8/4, 1-10/4 |
| Benchmark | Check rejects an equivalent answer on surface form (sign convention) | 1 | 1-6/4 |
| Both | Agent error, and the check is broken too (task is unpassable) | 4 | 1-3/4, 1-9/0, 1-14/2, 1-14/3 |
| Agent | Wrong operation / misread goal | 3 | 1-4/2, 1-5/2, 1-8/1 (+1-9/0, 1-14/2, 1-14/3 above) |
| Agent | Changed a label the task states literally | 1 | 1-4/3 |
| Agent | Ignored a conflict it had observed | 1 | 1-2/1 |
| Agent | Arithmetic: correct value found, then recomputed wrongly | 1 | 1-5/3 |
| Agent | Ran out of the 20-step budget (cell-by-cell writing) | 1 | 1-11/1 |

Five of the seven "check requires something else" cases (1-2/0, 1-2/2, 1-8/2, 1-8/4, 1-10/4) depend on how the task wording is read; the reading the check uses is defensible but not the only one. The other two (1-6/3, 1-8/0) are plain check errors.

### *What this does to the score*

| How the 21 failures are counted | Pass rate on the 73 scored |
|---|---|
| As scored by the benchmark | 52/73 = 71.2% |
| Credit the agent for the 5 clear benchmark defects (1-5/1, 1-6/0, 1-6/3, 1-6/4, 1-8/0) | 57/73 = 78.1% |
| Credit the agent for all 10 benchmark-side failures (adds the 5 wording-dependent cases) | 62/73 = 84.9% |

So the benchmark's own defects cost this run roughly 7 to 14 percentage points on the tasks scored so far. These are all single-app tasks; two- and three-app tasks have not started, so these numbers should not be compared

with the paper's overall leaderboard yet.

## Part 1 — Failures caused by the benchmark

In these cases the agent's final files (or its answer) match what the task asks. The run is scored as a failure because of the tool or the check.

### *B1. The Excel tool erases the value 1 (1-5/1, 1-6/0)*

**What happened.** In 1-5/1 ("reduce half amount of apple"), the agent read apple = 2, wrote **1** to cell (2,2), and the tool replied "Successfully write text". The saved file has that cell **empty**. In 1-6/0 ("randomly assign each student to class 1 to 5") the agent wrote 3, 1, 5, 2, 4. Every value was saved except the 1, so Carol's class is blank and the check fails.

**Why.** The action runs as a shell command, and the *fire* library turns the argument "1" into the integer 1. The first line of the tool then treats it as a boolean:

```
# apps/excel_app/excel_set_cell.py, line 36
def excel_set_cell(file_path, text, row_idx, column_idx, sheet_name=None):
    if text == True:     # 1 == True in Python, so the value 1 becomes ""
        text = ''
```

**Effect.** No agent can store the number 1 with *set_cell*, and the tool reports success, so the agent has no signal that anything went wrong. The same bug was present in the earlier no-thinking runs. The agent could have caught it by reading the file back after writing (it never did in these two runs). **Fix:** change the test to `text is True`.

### *B2. The check requires something the task does not ask for (7 subtasks)*

**1-2/0 and 1-2/2 — "find a time" is checked as "add an event".** The tasks say "Find a common time for Bob and Tom to have a one hour zoom meeting" and "help them find one hour" for shopping. The agent listed both calendars, worked out that Bob is free 14:00–16:00 and Tom is free 14:30–16:00, and answered **14:30–15:30**, which is correct. The checks require an event containing "meeting" / "shopping" in both .ics files, which the task never asks for. (The sister task 1-2/1 says "Add a common time on their calendar" explicitly, which suggests the authors meant this but left it out of 1-2/0 and 1-2/2.)

**1-6/3 — the check reads the wrong file.** The task says "show this in *salary* excel file with new header 'amount after 1 year'". The agent wrote the header and 210000 / 110000 / 110000 into salary.xlsx (base +5%, stock +10%, bonus +10%), all correct. Both checks open *company_budget.xlsx* and expect 2,100,000 / 55,000 / 1,100.

**1-8/0 — the reference deletes the student names.** Task: "delete all students' score and the header". The agent cleared the header row and every score and kept the names. The check compares the file cell by cell with *reference/score.xlsx*, which is completely empty; the only differences are the names Liam, Ivy, Alice. Note that the sister task 1-8/1 ("remove all students' class and the header") has a reference that *keeps* both the names and the "Name" header, so the two references read "the header" in opposite ways.

**1-8/2 and 1-8/4 — "delete all amounts" is checked as "delete the amount column".** The agent cleared the three amounts and kept the column header "amount". The reference also removes the header, so the single difference is cell (1,2) = "amount". A header is not an amount, so the agent's reading is at least as reasonable as the check's.

**1-10/4 — question asks for an amount, check wants a name.** "What is the fewest amount of bought item in shopping list excel file?" The agent read apple 2, cabbage 3, onion 4 and answered **2**. The check looks for the word "apple". Read literally, the question asks for the amount. An answer such as "apple (2)" would have satisfied both readings, so the agent could have hedged better here.

### *B3. A correct answer rejected on form (1-6/4)*

Task: add a header 'Increase Percentage Per Year', "increase base salaries by 5%/yr, **reduce** computer expenses 10%/yr, and increase entertainment expenses 10%/yr". The agent wrote **5%**, **-10%**, **10%**. The check expects 5, 10, 10. Because the cell checks are substring matches (see B5), the % sign is fine: "(2, 3): 5" is found inside "(2, 3): 5%". The run fails **only because of the minus sign** on the computer row, which is the correct way to write a reduction under a header called "Increase Percentage".

### *B4. Checks that no correct answer can pass*

Four further subtasks also involve an agent mistake (Part 2), but the check is broken as well, so they would fail even for a perfect agent:

- **1-14/2** — the check points to `reference/salery.xlsx` (misspelled). The file does not exist, so the check crashes with FileNotFoundError on every run.
- **1-14/3** — "swap the *fewest* and *second fewest* shopping item". The reference swaps apple (2) with *onion (4)*, the fewest and the most, so a correct swap of apple and cabbage can never match.
- **1-9/0** — the expected midterm1 average is "80.5", but the true mean of the 20 scores is **80.05**. Since "(22, 2): 80.5" is not a substring of "(22, 2): 80.05", the correct value fails.
- **1-3/4** — "move all docx, pdf, and image files", but one check also requires `file3.xlsx` to be in new_dir. Moving the spreadsheet would break the instruction; not moving it fails the check.

### *B5. How the checks work (why form and wording matter so much)*

Most Excel checks (`evaluate_excel_cell_value`) render the sheet as text and test whether the literal string `"(row, col): value"` appears anywhere in it. This has three consequences: (1) the answer must sit in exactly the expected cell; (2) form matters in odd ways: "08:00" fails against "8:00", "-10%" fails against "10", but "10%" passes; (3) some wrong answers pass, e.g. "80.55" would match "80.5". Answer checks (`evaluate_contain`) only look for a keyword anywhere in answer.txt. The "exact match" checks compare every cell with a reference file, so any disagreement over what to keep (names, headers) is fatal.

## Part 2 — Failures caused by the agent

These 11 runs (7 agent-only plus the 4 from B4) contain a real model error. For each one the decisive step is the earliest step that, if corrected alone, would have made the output right.

### *A1. Doing a different operation from the one asked (6 subtasks)*

This is the largest agent-side group. The model reads the data correctly and does valid arithmetic, but on the wrong target.

- **1-4/2 — merged instead of added.** "add annual bonus $50000 to salary excel file". The sheet already has a row "bonus 100000". The agent reasoned "the new bonus amount should be 100,000 + 50,000 = 150,000" and overwrote that cell. Expected: a new row 5, "annual bonus" / 50000. (The same mistake appeared in the earlier run.)
- **1-5/2 — ignored "for each day".** "add 'average salary' for each day ... round down". The agent wrote 133333, the mean of the three salary lines (base, stock, bonus), saying the phrase "in this context seems to be the average of the amounts listed". Expected: total 400000 / 365 = 1095.
- **1-9/0 — row averages instead of a bottom row.** "calculate the average of two midterm scores, write in the bottom row ... with header 'Average'". The agent added a new *column* with each student's average of their two scores, one cell per step, and hit the 20-step limit at row 15. Expected: row 22 = "Average", mean of midterm1, mean of midterm2. (The expected value is also wrong; see B4.)
- **1-14/2 and 1-14/3 — swapped the labels, not the items.** Asked to swap two items, the agent rewrote only the name cells and left the amounts in place. In 1-14/3 the sheet now says cabbage = 2 and apple = 3, which changes the data instead of moving it; in 1-14/2 "stock" is now 200000. The reference swaps whole rows. The model did swap whole rows in the passing sister task 1-14/4, so this is inconsistent behaviour rather than a fixed belief.
- **1-8/1 — deleted one header too many.** "remove all students' class and the header". The agent cleared the Class column and its header, read the file back, then decided "the header" also meant "Name" and deleted it too. The reference keeps "Name". The wording is ambiguous, and the sister reference for 1-8/0 reads it the other way.

### *A2. Changing a literal label (1-4/3)*

“add utilities fee $300 to budget excel file”. The agent correctly added a new row 5 with 300, but in its thought shortened the label: “I should add ‘utilities’ to the first column”. The check wants the task’s exact words, “utilities fee”. Same error as in the earlier run.

### *A3. Ignoring a conflict it had seen (1-2/1)*

“Can Bob and Tom have dinner together at 5/1/2024? Add a common time on their calendar.” Both calendars already contain “dinner 18:30–19:30”. The agent noticed this:

> *“Bob’s dinner is from 18:30 to 19:30. Tom’s dinner is also from 18:30 to 19:30 ... they already have the same time slot occupied by ‘dinner’. To be explicit and ensure it’s a shared event, I ...” (step 3)*

It then added a second event, “Dinner with Tom/Bob 18:30–19:30”, to each calendar, so the no-overlap check fails for both. Answering “yes” without adding anything, or booking a free slot, would have passed.

### *A4. Correct arithmetic, then recomputed wrongly (1-5/3)*

“if budget grow 10% per year, add ‘predicted budget’ for 3 years later”. At step 4 the model worked it out year by year and got the right answer:

> *“Year 1: 2,051,000 × 1.1 = 2,256,100. Year 2: 2,481,710. Year 3: 2,481,710 × 1.1 = **2,729,881**.” (step 4, while writing the label)*

At step 5, writing the number, it worked it out again a different way and slipped:

> *“2,051,000 × 1.331 = **2,730,381**.” (step 5)*

It wrote 2730381. **The root cause is in the harness:** each prompt carries the history of actions and observations but *not* the model’s earlier thoughts, so a value worked out in one step is lost and must be recomputed at the next. Every multi-step write in this run starts its thought by re-deriving the plan (“I have already ...”), which costs tokens and gives the model a fresh chance to make an arithmetic mistake.

### *A5. Running out of steps (1-11/1, and 1-9/0)*

“split midterm1 and midterm2 scores ... into two excel files”. The agent built both files cell by cell: create file, two headers, then name + score for each of 4 students. That is 11 actions per file plus 4 actions of setup, about 26 in total, against a hard limit of 20. midterm1.xlsx came out exactly right; midterm2.xlsx had only its header and Liam when the run was cut off. The plan was correct but could not fit. Copying score.xlsx in the shell and deleting one column, or a single shell command that writes both files, would have fit. The same budget limit stopped 1-9/0 at row 15 of 21.

### *A6. Blind shell commands (1-3/4)*

“move all docx, pdf, and image files to a new subdirectory named new_dir”. The thought says “First, I will list the files”, but it never did. It ran `mkdir -p new_dir` and `mv *.docx *.pdf *.jpg ... new_dir/ 2>/dev/null || true` with relative paths. The shell starts in `/` (WORKDIR in docker/Dockerfile), not /testbed/data, so the directory was made at the root, nothing matched, and the error output was thrown away by the agent’s own `2>/dev/null || true`. The tool reported success and the agent finished. In every other task this run used absolute /testbed/data paths, and the earlier run solved this task correctly, so this is a regression and a matter of luck rather than a capability gap.

## Part 3 — Patterns across the failures

- **The agent never checks its writes.** It almost never reads a file back after writing to it (only in the 1-8 delete tasks). One read after writing would have revealed the erased “1” in 1-5/1 and 1-6/0, the empty folder in 1-3/4, and the double booking in 1-2/1.
- **Earlier thoughts are not carried forward.** Because the prompt keeps only actions and observations, the model re-plans at every step. This caused the arithmetic error in 1-5/3 and uses up tokens on every long task.
- **Excel actions are one cell at a time, with a 20-step budget.** Any task that writes more than about 14 cells cannot finish with *set_cell* alone. The model does not switch to a bulk method on its own.

- **Loose reading of the instruction.** The model tends to choose the nearest easy operation: updating an existing row instead of adding one, averaging what is in front of it, shortening a label, renaming instead of moving. The thinking prompt did not remove this; the same errors on 1-4/2, 1-4/3 and 1-5/2 appear in the earlier no-thinking run.
- **What thinking did fix.** Compared with the earlier run, malformed or looping actions have almost disappeared (3 malformed steps in all 73 runs, in 1-9/0 and 1-9/1, and the agent recovered from each), and 1-6/2 now passes. The failures that remain are about meaning, not format.

# Part 4 — What to fix

### *Benchmark fixes (each one changes the score of a correct agent)*

| Where | Problem | Fix | Affects |
|---|---|---|---|
| apps/excel_app/excel_set_cell.py :36 | `text == True` erases the value 1 | `text is True` | 1-5/1, 1-6/0 and any task writing 1 |
| tasks/1-14/subtasks/2.json | expected_file is "salery.xlsx" | rename to salary.xlsx | 1-14/2 |
| tasks/1-14/reference/shopping_list.xlsx | swaps fewest with most | swap apple and cabbage rows | 1-14/3 |
| tasks/1-9/subtasks/0.json | expects 80.5; true mean is 80.05 | expect 80.05 | 1-9/0 |
| tasks/1-6/subtasks/3.json | checks company_budget.xlsx | check salary.xlsx, 210000 / 110000 / 110000 | 1-6/3 |
| tasks/1-3/subtasks/4.json | requires file3.xlsx in new_dir | drop that check (or check it stayed) | 1-3/4 |
| tasks/1-8/reference/score.xlsx | names deleted | keep the Name column | 1-8/0 |
| tasks/1-2/subtasks/0.json, 2.json | "find" checked as "add event" | add "and add it to their calendars" to the task, or check the answer | 1-2/0, 1-2/2 |
| history_to_trace.py evaluate() | reads task JSON as cp1252 | open(..., encoding='utf-8') | 1-16/0 (dashboard only) |

The wording-dependent cases (1-8/2, 1-8/4, 1-10/4, and 1-8/1 on the agent side) are better reported as ambiguous than fixed one way; if they are changed, the two 1-8 references should at least agree on what “the header” means.

### *Agent / harness changes worth testing*

- **Carry the last thought (or a short “notes” field) into the next prompt**, so computed values and plans survive between steps. Targets 1-5/3 and reduces re-planning on every long task.
- **Add a verify rule to the prompt:** after writing, read the file back before finish_task. Targets 1-5/1, 1-6/0, 1-3/4, 1-2/1.
- **Tell the agent about bulk options** (copy a file in the shell, or one shell command that writes many cells) and that the step budget is 20. Targets 1-11/1, 1-9/0.
- **Prompt rules for literal reading:** use labels exactly as written in the task; “add X” means a new row; a swap moves whole rows; for a question, give both the item and the value. Targets 1-4/2, 1-4/3, 1-14/2, 1-14/3, 1-10/4.
- **Use absolute paths in the shell and never silence errors** with `2>/dev/null || true`. Targets 1-3/4.

# Appendix — all 21 failures

Code = primary code from analysis/codebook.md. Side: B = benchmark, A = agent, A+B = agent error on a broken check. “(ambig.)” marks a task whose wording supports more than one reading.

| ID | Task (short) | What the agent did | What the check wanted | Code | Side |
|---|---|---|---|---|---|
| **1-2/0** | Find a common 1-hour zoom slot for Bob and Tom | Answered 14:30-15:30 (correct); added no event | "meeting" event in both calendars | EVAL-MISMATCH (ambig.) | B |
| **1-2/1** | Can they have dinner together? Add a common time | Saw both already have dinner 18:30-19:30; added another at the same time | No overlapping events | A-STATE | A |
| **1-2/2** | Help Bob and Tom find one hour to shop | Answered 14:30-15:30 (correct); added no event | "shopping" event in both calendars | EVAL-MISMATCH (ambig.) | B |
| **1-3/4** | Move docx, pdf, image files to new_dir | Relative-path mkdir/mv in "/" with errors hidden; nothing moved | docx, pdf, jpg *and xlsx* in new_dir | A-REASON + broken check | A+B |
| **1-4/2** | Add annual bonus $50000 | Overwrote bonus 100000 to 150000 | New row "annual bonus" / 50000 | A-INTENT | A |
| **1-4/3** | Add utilities fee $300 | New row "utilities" / 300 | "utilities fee" / 300 | A-DEVIATION | A |
| **1-5/1** | Halve the apples | Wrote 1; tool saved an empty cell | (2,2) = 1 | ENV-TOOL | B |
| **1-5/2** | Average salary for each day, round down | 133333 (mean of 3 rows) | 1095 (400000 / 365) | A-INTENT | A |
| **1-5/3** | Budget after 3 years at 10% | Found 2729881, then rewrote it as 2730381 | 2729881 | A-REASON | A |
| **1-6/0** | Add Class header, random class 1-5 | Wrote 3,1,5,2,4; the 1 was erased | Each row in 1-5 | ENV-TOOL | B |
| **1-6/3** | Raises shown in salary file | Correct values in salary.xlsx | Values in company_budget.xlsx | EVAL-MISMATCH | B |
| **1-6/4** | Increase % per year header | 5%, -10%, 10% | 5, 10, 10 (minus sign rejected) | EVAL-FORM | B |
| **1-8/0** | Delete scores and the header | Cleared header row and scores; kept names | Empty sheet (names deleted too) | EVAL-MISMATCH | B |
| **1-8/1** | Remove classes and the header | Cleared Class column, then also deleted "Name" | Keep "Name" | A-INTENT (ambig.) | A |
| **1-8/2** | Delete all amounts (salary) | Cleared amounts; kept "amount" header | Header deleted too | EVAL-MISMATCH (ambig.) | B |
| **1-8/4** | Delete all amounts (shopping) | Cleared amounts; kept "amount" header | Header deleted too | EVAL-MISMATCH (ambig.) | B |
| **1-9/0** | Average of two midterms in bottom row | Per-student average column; hit 20 steps at row 15 | Row 22: Average / 80.5 / 78.3 (80.5 is wrong; true 80.05) | A-INTENT + broken check | A+B |
| **1-10/4** | What is the fewest amount of bought item? | Answered "2" | Answer contains "apple" | EVAL-MISMATCH (ambig.) | B |
| **1-11/1** | Split scores into midterm1/2.xlsx | midterm1 correct; midterm2 cut off at 20 steps | Both files exact | A-ACTION (budget) | A |
| **1-14/2** | Swap lowest and highest salary item | Swapped names only; amounts left in place | reference "salery.xlsx" does not exist (crash) | A-INTENT + broken check | A+B |
| **1-14/3** | Swap fewest and second fewest item | Swapped names only (cabbage 2, apple 3) | Reference swaps apple with onion | A-INTENT + broken check | A+B |

## *Method*

Source data: tasks/<task>/outputs/<sub>/gemma4_31b_think/ (env_history.json, llm_history.json, final testbed), the reference files under tasks/<task>/reference/, and the check code in utils/evaluate.py. Every check was re-run with UTF-8 file reading. Each failure was given one primary code following the decision procedure in analysis/codebook.md (judge the output against the task as written; if it is right, the cause is on the benchmark side; if it is wrong, code the decisive agent step). These codes were assigned by one annotator from the traces. Unlike the earlier 16-case set (two annotators, $\kappa = 0.78$ on primary code), they have not been double-coded, and the wording-dependent cases are the ones most likely to be coded differently by a second annotator.